

Git — Basic Notes

1. What is Git?

Git is a **Distributed Version Control System (DVCS)** used to track changes and maintain the history of files and projects.

2. Version Control System

A **Version Control System (VCS)** helps us:

- Track changes made to files.
- Maintain the history of a project.
- Go back to previous versions.
- Collaborate with other developers.
- Identify who made changes and when.

Why Git?

1. Maintains Project History

- We can see what changes were made and when.
- We can restore previous versions of the project.
- We can compare different versions of the project.

2. Helps with Collaboration

- Multiple developers can work on the same project.
- Developers can share their changes.
- Developers can merge their changes.

3. Distributed

Every developer can have a complete copy of the repository and its history on their local machine.

3. Repository

A **repository (repo)** is a project directory that is managed and tracked by Git.

When we run:

```
git init
```

inside a project directory, Git initializes that directory as a **Git repository**.

Git creates a hidden `.git` directory that contains the information Git needs to track the project.

Example

```
my-project/  
├── .git/  
├── index.html  
├── app.js  
└── style.css
```

The `.git` directory is the core of a Git repository. Do not manually modify or delete it.

4. Basic Git Workflow

```
Working Directory  
  ↓  
git add  
  ↓  
Staging Area  
  ↓  
git commit  
  ↓  
Local Repository  
  ↓  
git push  
  ↓  
Remote Repository
```

In simple terms:

```
Make changes  
  ↓  
git add  
  ↓  
Select changes for commit  
  ↓  
git commit  
  ↓  
Save a version in Git history  
  ↓  
git push  
  ↓  
Upload commits to remote repository
```

5. Git Configuration

Before making commits, configure your Git identity.

Configure Username

```
git config --global user.name "Your Name"
```

Example:

```
git config --global user.name "Tarun Pandey"
```

Configure Email

```
git config --global user.email "your-email@example.com"
```

Example:

```
git config --global user.email "tarun@example.com"
```

Check Username

```
git config --global user.name
```

Check Email

```
git config --global user.email
```

View Global Configuration

```
git config --global --list
```

View All Configuration

```
git config --list
```

6. Initialize a Repository

```
git init
```

Initializes a Git repository.

```
git init
```

Example:

```
mkdir my-project  
cd my-project  
git init
```

Git creates:

```
my-project/  
└─ .git/
```

7. Check Repository Status

```
git status
```

Checks the current state of the repository.

```
git status
```

It tells us:

- Which files are untracked.
- Which files have been modified.
- Which files are staged.
- Which files are ready to be committed.

Example:

```
touch app.js  
git status
```

`app.js` will appear as an **untracked file**.

8. Git Add

```
git add
```

Moves changes from the **Working Directory** to the **Staging Area**.

```
Working Directory
  ↓
git add
  ↓
Staging Area
```

Add a Specific File

```
git add app.js
```

Add Multiple Files

```
git add app.js index.js
```

Add All Changes

```
git add .
```

`git add` does **not** create a commit. It only stages changes.

9. Git Commit

```
git commit
```

Creates a snapshot of the staged changes.

```
git commit -m "Add login functionality"
```

Typical Workflow

```
git add .
git commit -m "Add user authentication"
```

A commit message should clearly describe the change.

Good:

```
git commit -m "Add user authentication"
```

Not recommended:

```
git commit -m "changes"
```

10. Git Log

```
git log
```

Displays commit history.

```
git log
```

It shows:

- Commit ID
- Author
- Date
- Commit message

Compact History

```
git log --oneline
```

Example:

```
a82f91c Add login functionality  
7b32c1a Create project structure
```

11. GitHub

GitHub is a platform for hosting Git repositories and collaborating with other developers.

Git and GitHub are **not the same thing**.

Git

```
Git
↓
Version Control System
↓
Runs on your computer
```

GitHub

```
GitHub
↓
Remote Repository Hosting
↓
Collaboration Platform
```

12. Local Repository vs Remote Repository

Local Repository

Stored on your computer.

```
Your Computer
├─ my-project
│   └─ .git
```

Remote Repository

Stored on a remote Git hosting platform.

```
Remote Server
├─ my-project
```

13. Add a Remote Repository

```
git remote add
```

Connects a local repository to a remote repository.

Syntax

```
git remote add origin <remote-repository-url>
```

Example

```
git remote add origin https://github.com/username/my-project.git
```

Here:

```
origin
```

is the name given to the remote repository.

14. Check Remote Repository

```
git remote -v
```

Example:

```
origin https://github.com/username/my-project.git (fetch)
origin https://github.com/username/my-project.git (push)
```

15. Personal Access Token (PAT)

When using HTTPS authentication with GitHub, a **Personal Access Token (PAT)** can be used instead of a traditional account password.

A PAT should be treated like a password.

Important

Never:

- Share your token.
- Commit the token to Git.
- Put the token inside source code.
- Put the token directly into a remote URL.
- Share the token in screenshots or messages.

If a token is accidentally exposed, **revoke it and create a new token**.

16. Git Credential Manager

When Git communicates with a remote repository over HTTPS, it needs credentials to authenticate you.

Git can use a **credential helper** to save and retrieve those credentials.

One commonly used helper is:

```
Git Credential Manager (GCM)
```

Git Credential Manager can securely store credentials and allow Git to reuse them instead of asking for credentials repeatedly.

The exact storage mechanism depends on the operating system.

Typical secure storage includes:

```
Windows
  ↓
Windows Credential Manager

macOS
  ↓
macOS Keychain

Linux
  ↓
Configured credential manager / secret service
```

The exact behavior depends on your Git and Git Credential Manager installation.

17. Check Which Credential Helper Git Uses

Run:

```
git config --show-origin --get-all credential.helper
```

You can also check the configured credential helper with:

```
git config --global credential.helper
```

If a helper is configured, Git can use it to store/retrieve credentials.

18. Common Credential Storage Methods

Git can use different credential helpers.

1. Git Credential Manager

Recommended for many desktop setups because it integrates with the operating system's secure credential storage.

```
git config --global credential.helper manager
```

On some installations, the helper may be configured automatically.

2. `osxkeychain` — macOS

macOS can use the system Keychain.

```
git config --global credential.helper osxkeychain
```

Credentials are stored in the macOS Keychain rather than in a normal text file.

3. `store`

Git can permanently store credentials in a file.

```
git config --global credential.helper store
```

The credentials are typically stored in:

```
~/.git-credentials
```

Important Security Warning

The `store` helper stores credentials **unencrypted**.

Therefore:

```
credential.helper store
    ↓
~/.git-credentials
    ↓
Plain-text credentials
```

It should generally be avoided on shared or untrusted computers.

4. `cache`

Git can temporarily keep credentials in memory.

```
git config --global credential.helper cache
```

By default, the credentials are cached for a limited amount of time.

You can specify the timeout:

```
git config --global credential.helper "cache --timeout=3600"
```

`3600` means approximately **1 hour**.

The cache is temporary and is not intended for permanent credential storage.

19. Credential Helper Comparison

Method	Storage	Persistent?	Security
Git Credential Manager	OS credential store	Yes	Recommended
<code>osxkeychain</code>	macOS Keychain	Yes	Recommended on macOS
<code>store</code>	<code>~/.git-credentials</code>	Yes	Plain text
<code>cache</code>	Memory	No	Temporary

Recommendation

For a personal development machine:

```
Git Credential Manager
  ↓
OS Secure Credential Store
```

is generally preferable to storing credentials as plain text.

20. What Happens When Git Asks for a Password?

Suppose you run:

```
git push origin main
```

Git may ask:

```
Username for 'https://github.com':
Password for 'https://github.com':
```

For GitHub HTTPS authentication:

```
Username
  ↓
Your GitHub username

Password
  ↓
Your Personal Access Token
```

The PAT is used in the **password field**.

Your GitHub account password is not the value you should normally enter there for Git HTTPS authentication.

21. If Git Keeps Asking for the Password

First check whether a credential helper is configured:

```
git config --show-origin --get-all credential.helper
```

If nothing is returned, configure an appropriate credential helper.

For example, with Git Credential Manager:

```
git config --global credential.helper manager
```

Then try:

```
git push
```

Git may ask for authentication again.

After successful authentication, the credential manager can store the credential for future use.

22. If the Stored Credential Is Wrong

Sometimes Git keeps using an old or invalid token.

For example:

```
Old Token
  ↓
Credential Manager
  ↓
git push
  ↓
Authentication Failed
```

In this situation, remove the old credential from the operating system's credential store or credential manager, then run:

```
git push
```

Git should ask for authentication again.

Enter:

```
Username → Your Git hosting username
Password → New Personal Access Token
```

23. Forget Credentials Using Git Credential Commands

Git can also interact with the configured credential helper through:

```
git credential
```

For example, to request Git to erase stored credentials for a host:

```
printf "protocol=https\nhost=github.com\n\n" | git credential reject
```

After this, the next authentication attempt may ask for credentials again.

Whether the credential is actually removed depends on the configured credential helper.

24. Check Credential Configuration

Use:

```
git config --show-origin --get-all credential.helper
```

To check global configuration:

```
git config --global --list
```

To check repository-specific configuration:

```
git config --local --list
```

This is useful when Git behaves differently for different repositories.

25. Recommended Setup for Students

For a normal personal computer, a simple approach is:

Step 1 — Configure Git Identity

```
git config --global user.name "Your Name"  
git config --global user.email "your-email@example.com"
```

Step 2 — Check Credential Helper

```
git config --show-origin --get-all credential.helper
```

Step 3 — Use Git Credential Manager if available

```
git config --global credential.helper manager
```

Step 4 — Add GitHub Remote

```
git remote add origin https://github.com/username/my-project.git
```

Step 5 — Push

```
git push -u origin main
```

Step 6 — Authenticate

If prompted:

```
Username → GitHub username  
Password → Personal Access Token
```

Step 7 — Future Pushes

After the credential has been successfully stored:

```
git push
```

may no longer ask for credentials.

26. Important Security Rules

Never Store a Token in the Repository

Do **not** do this:

```
project/  
├─ app.js  
├─ package.json  
└─ token.txt
```

Do not commit credentials.

Never Put a Token Directly in the Remote URL

Avoid:

```
git remote add origin https://username:TOKEN@github.com/username/project.git
```

The token can be exposed through shell history, configuration, logs, or other tools.

Use:

```
git remote add origin https://github.com/username/project.git
```

and allow Git's authentication mechanism to handle credentials.

27. If a Token Is Accidentally Exposed

Immediately:

1. Revoke the exposed token.
2. Create a new token.
3. Remove the token from the exposed location.
4. Check Git history if the token was committed.
5. Update the credential stored by your credential manager.

Simply deleting a token from the latest version of a file does not necessarily remove it from Git history.

28. First-Time GitHub Push

```
mkdir my-project
cd my-project

git init

touch app.js

git add .

git commit -m "Initial commit"

git branch -M main

git remote add origin https://github.com/username/my-project.git

git push -u origin main
```

If Git asks for authentication:

```
Username:
  Your GitHub username

Password:
  Your Personal Access Token
```

After successful authentication, the configured credential helper may remember the credentials.

29. Git Workflow with GitHub

```
Local Computer
  |
  v
Working Directory
  |
  v
git add
  |
  v
Staging Area
  |
  v
git commit
  |
  v
Local Repository
  |
  v
git push
  |
  v
```

```
Remote Repository
  (GitHub)
    ↑
  git pull
    ↑
Remote Changes
```

30. Git Authentication — Quick Revision

Git HTTPS Authentication

```
↓
Credential Helper
  ↓
├── GCM
├── Keychain
└── Cache
    ↓
  OS Secure Storage
```

Common options

```
# Git Credential Manager
git config --global credential.helper manager

# macOS Keychain
git config --global credential.helper osxkeychain

# Temporary in-memory cache
git config --global credential.helper cache

# Plain-text storage – generally avoid
git config --global credential.helper store
```

Check current helper

```
git config --show-origin --get-all credential.helper
```

If Git asks for credentials

```
Username → GitHub username
Password → Personal Access Token
```

If Git repeatedly asks

1. Check `credential.helper`
2. Check whether an old credential is stored
3. Remove/reject the old credential
4. Run `git push` again
5. Authenticate with the new token

31. Complete Basic Git Command Summary

Command	Purpose
<code>git config --global user.name "Name"</code>	Configure username
<code>git config --global user.email "Email"</code>	Configure email
<code>git config --global --list</code>	View global configuration
<code>git init</code>	Initialize repository
<code>git status</code>	Check repository status
<code>git add <file></code>	Stage a specific file
<code>git add .</code>	Stage changes
<code>git commit -m "message"</code>	Create a commit
<code>git log</code>	View commit history
<code>git log --oneline</code>	View compact history
<code>git branch</code>	List branches
<code>git switch <branch></code>	Switch branches
<code>git switch -c <branch></code>	Create and switch branch
<code>git remote add origin <url></code>	Add remote repository
<code>git remote -v</code>	View remote repositories
<code>git clone <url></code>	Clone repository
<code>git fetch</code>	Download remote changes
<code>git pull</code>	Download and integrate remote changes
<code>git push</code>	Upload commits
<code>git push -u origin main</code>	Push and set upstream

Command	Purpose
<code>git config --show-origin --get-all credential.helper</code>	Check credential helper

32. Key Concepts to Remember

Term	Meaning
Git	Distributed Version Control System
Repository	Project tracked by Git
<code>.git</code>	Directory containing Git repository data
Working Directory	Files currently being worked on
Staging Area	Changes selected for the next commit
Commit	Snapshot of staged changes
Branch	Separate line of development
Local Repository	Repository on your computer
Remote Repository	Repository hosted remotely
origin	Common name for the primary remote
GitHub	Platform for hosting Git repositories
PAT	Personal Access Token used for authentication
Credential Helper	Mechanism used by Git to store/retrieve credentials
Git Credential Manager	Credential helper that integrates with secure OS credential storage
SSH	Key-based authentication method

Final Recommended Workflow

```

1. Install Git
   ↓
2. Configure username and email
   ↓
3. Create project
   ↓
4. git init
   ↓
5. Make changes
   ↓
6. git add .

```

- ↓
- 7. `git commit -m "message"`
- ↓
- 8. Create repository on GitHub
- ↓
- 9. `git remote add origin <URL>`
- ↓
- 10. `git push -u origin main`
- ↓
- 11. Authenticate using PAT/credential manager
- ↓
- 12. Future `git push` / `git pull`